ClinicConnect Developer Guide

- ClinicConnect Developer Guide
  - Acknowledgements
  - Setting up, getting started
  - Design
    - Architecture
    - UI component
    - Logic component
    - Model component
    - Storage component
    - Common classes
  - Implementation
    - Automatic Triage Sorting
    - List feature
    - Find feature
    - Undo Feature
    - Command History Navigation
  - Documentation, logging, testing, configuration, dev-ops
  - Appendix: Requirements
    - Product scope
    - User stories
    - Use cases
    - Non-Functional Requirements
    - Glossary
  - Appendix: Instructions for manual testing
    - Launch and shutdown
    - Adding a person
    - Deleting patient(s)
    - Updating patient(s)
    - Saving data
  - Appendix: Planned Enhancements

Acknowledgements

- This project is based on the AddressBook Level-3 project created by the SE-EDU initiative.

- Libraries used: JavaFX, Jackson, JUnit5.
- AI was used in this following manner:
  - Randy
    - Used Co-pilot for auto-completion.
    - Used ChatGPT to assist in generating regex pattern for user input validation.
  - Nigel
    - Used Gemini to assist with debugging logic errors in command execution and parser architecture.
    - Used Gemini to assist in generating test cases for bulk updates

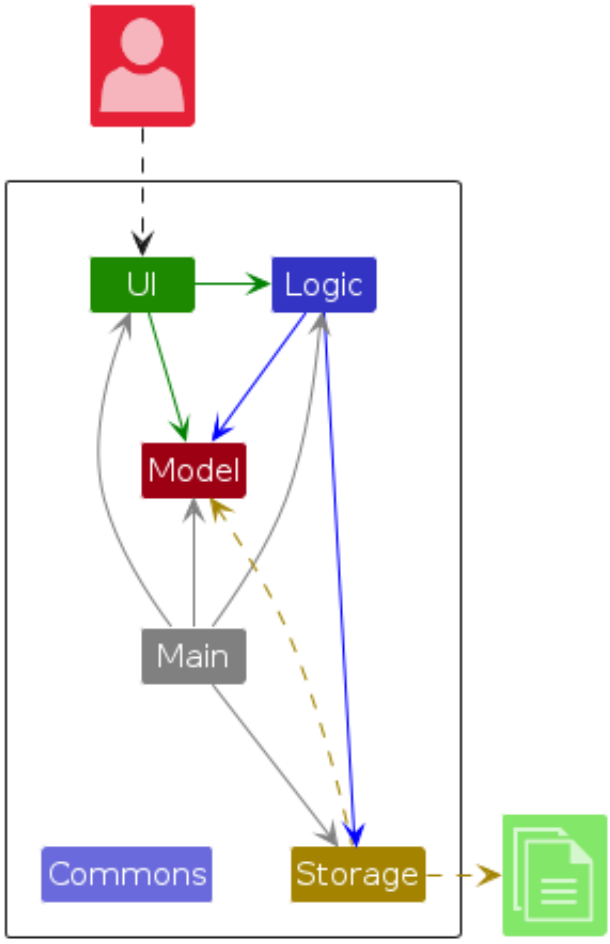
---

Setting up, getting started

Refer to the guide *Setting up and getting started*.

Design

Architecture



The ***Architecture Diagram*** given above explains the high-level design of the App.

Given below is a quick overview of main components and how they interact with each other.

Main components of the architecture

`Main` (consisting of classes `Main` and `MainApp`) is in charge of the app launch and shut down.

- At app launch, it initializes the other components in the correct sequence, and connects them up with each other.
- At shut down, it shuts down the other components and invokes cleanup methods where necessary.

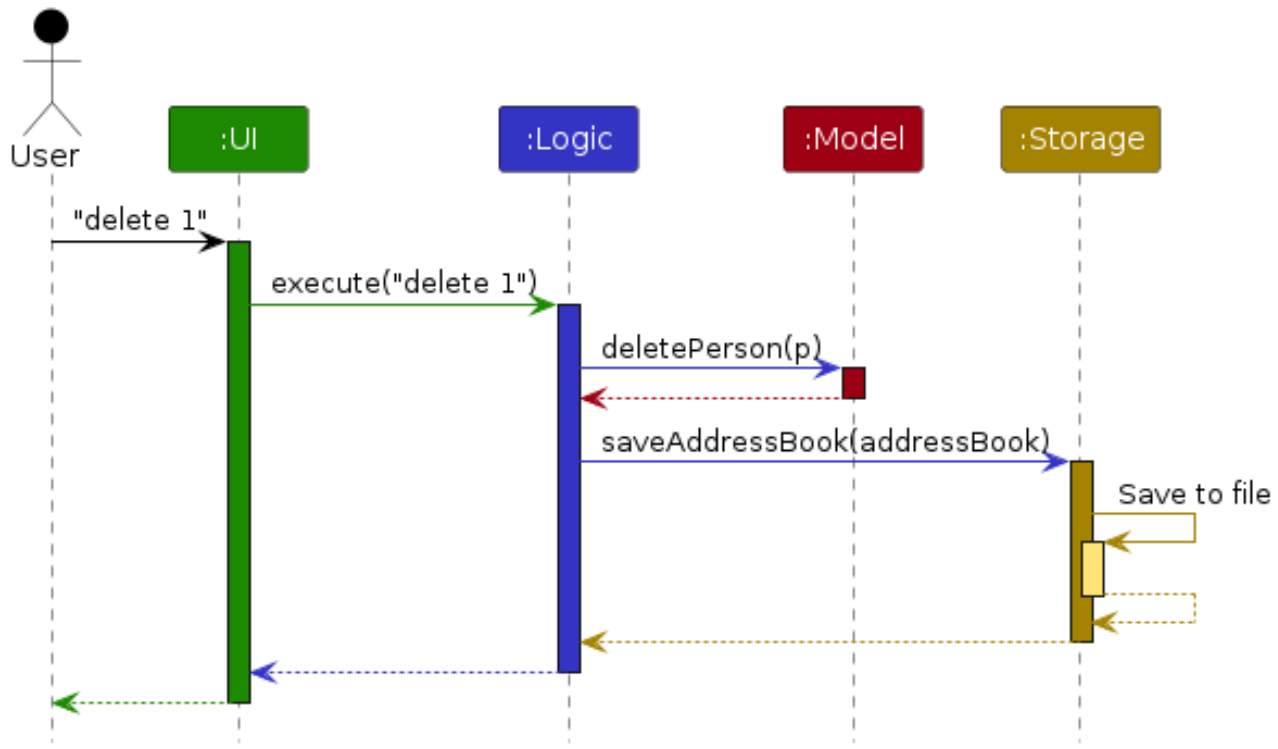
The bulk of the app's work is done by the following four components:

- `UI` : The UI of the App.
- `Logic` : The command executor.
- `Model` : Holds the data of the App in memory.
- `Storage` : Reads data from, and writes data to, the hard disk.

`Commons` represents a collection of classes used by multiple other components.

How the architecture components interact with each other

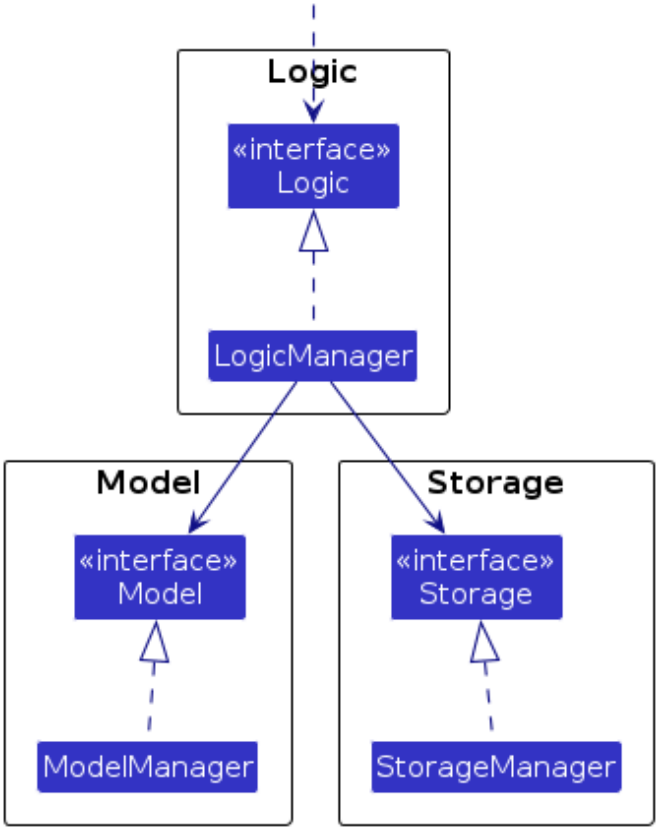
The *Sequence Diagram* below shows how the components interact with each other for the scenario where the user issues the command `delete 1`.



Each of the four main components (also shown in the diagram above),

- defines its *API* in an `interface` with the same name as the Component.
- implements its functionality using a concrete `{Component Name}Manager` class (which follows the corresponding API `interface` mentioned in the previous point.

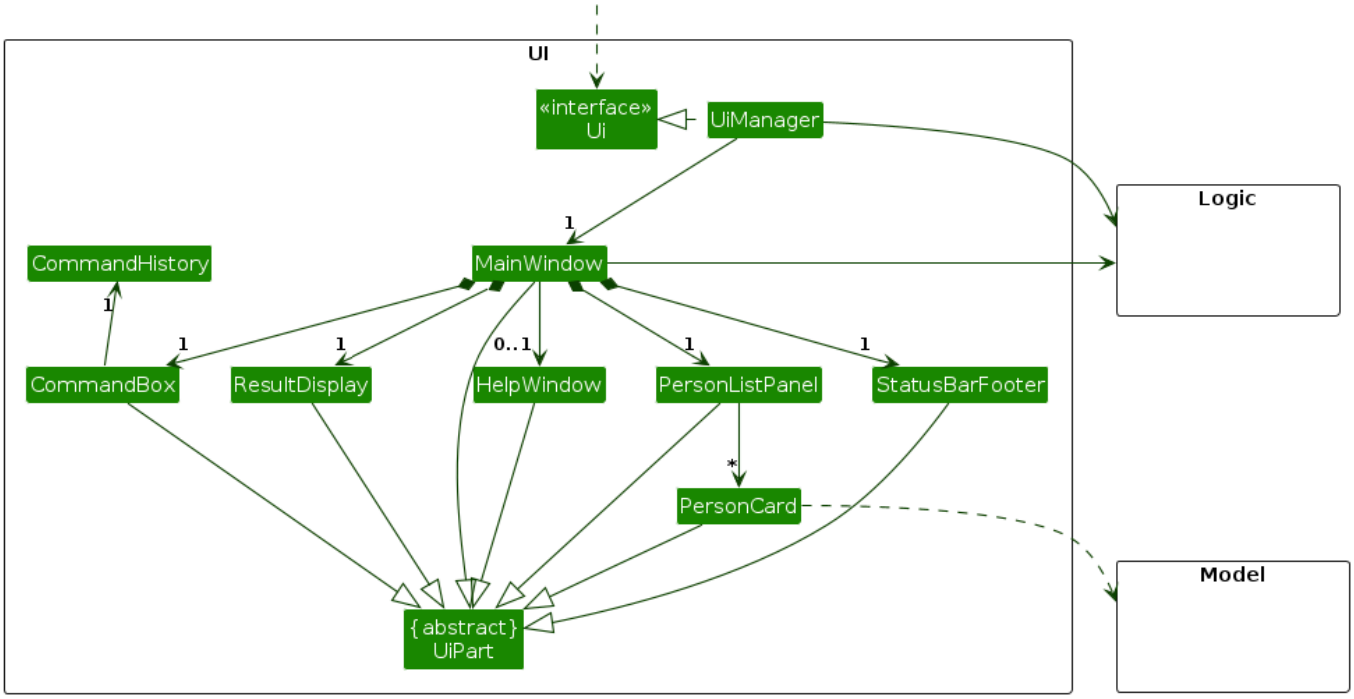
For example, the `Logic` component defines its API in the `Logic.java` interface and implements its functionality using the `LogicManager.java` class which follows the `Logic` interface. Other components interact with a given component through its interface rather than the concrete class (reason: to prevent outside component's being coupled to the implementation of a component), as illustrated in the (partial) class diagram below.



The sections below give more details of each component.

UI component

The **API** of this component is specified in `Ui.java`



The UI consists of a `MainWindow` that is made up of parts e.g. `CommandBox`, `ResultDisplay`, `PersonListPanel`, `StatusBarFooter` etc. All these, including the `MainWindow`, inherit from the abstract `UiPart` class which captures the commonalities between classes that represent parts of the visible GUI.

The `UI` component uses the JavaFx UI framework. The layout of these UI parts are defined in matching `.fxml` files that are in the `src/main/resources/view` folder. For example, the layout of the `MainWindow` is specified in `MainWindow.fxml`

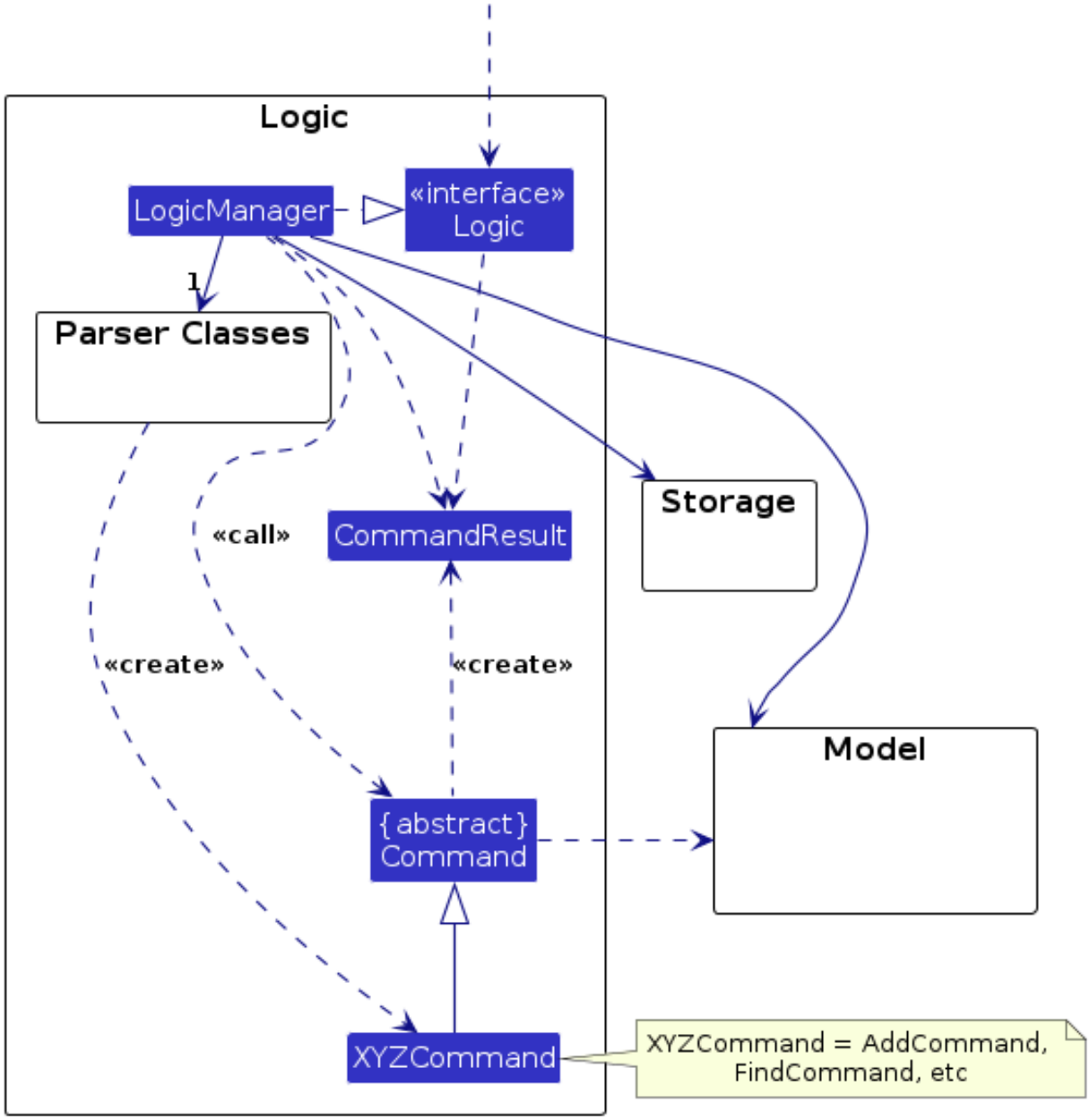
The `UI` component,

- executes user commands using the `Logic` component.
- listens for changes to `Model` data so that the UI can be updated with the modified data.
- keeps a reference to the `Logic` component, because the `UI` relies on the `Logic` to execute commands.
- depends on some classes in the `Model` component, as it displays `Person` object residing in the `Model`.

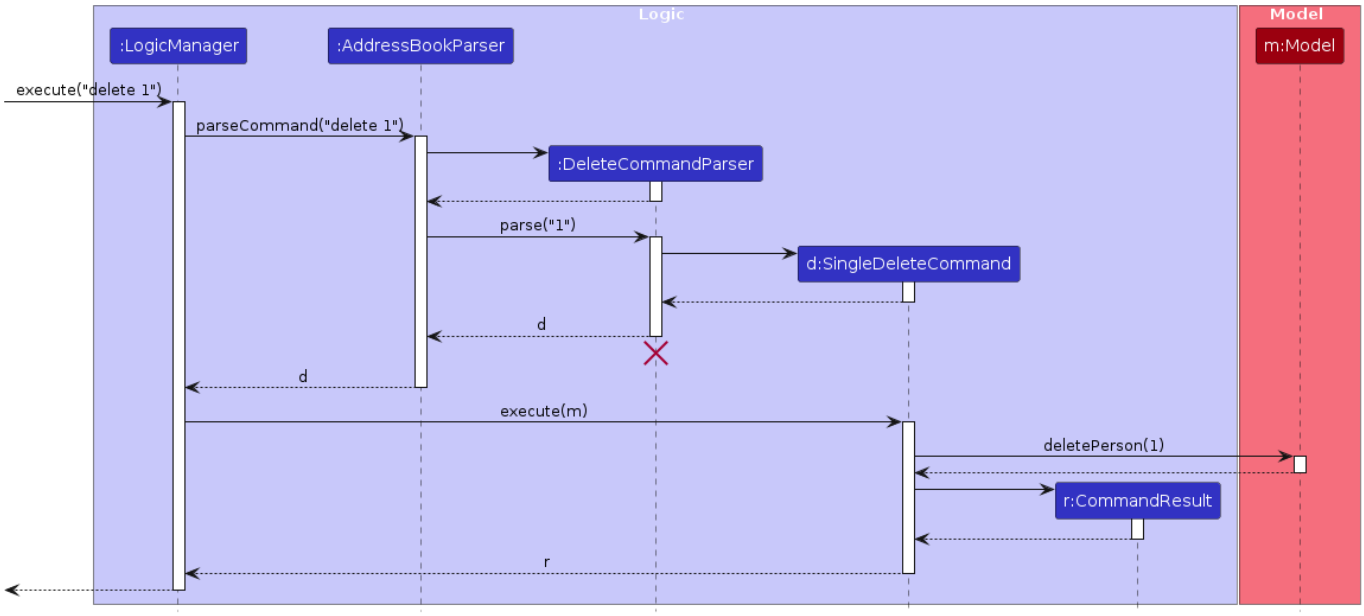
Logic component

API : `Logic.java`

Here's a (partial) class diagram of the `Logic` component:



The sequence diagram below illustrates the interactions within the `Logic` component, taking `execute("delete 1")` API call as an example.

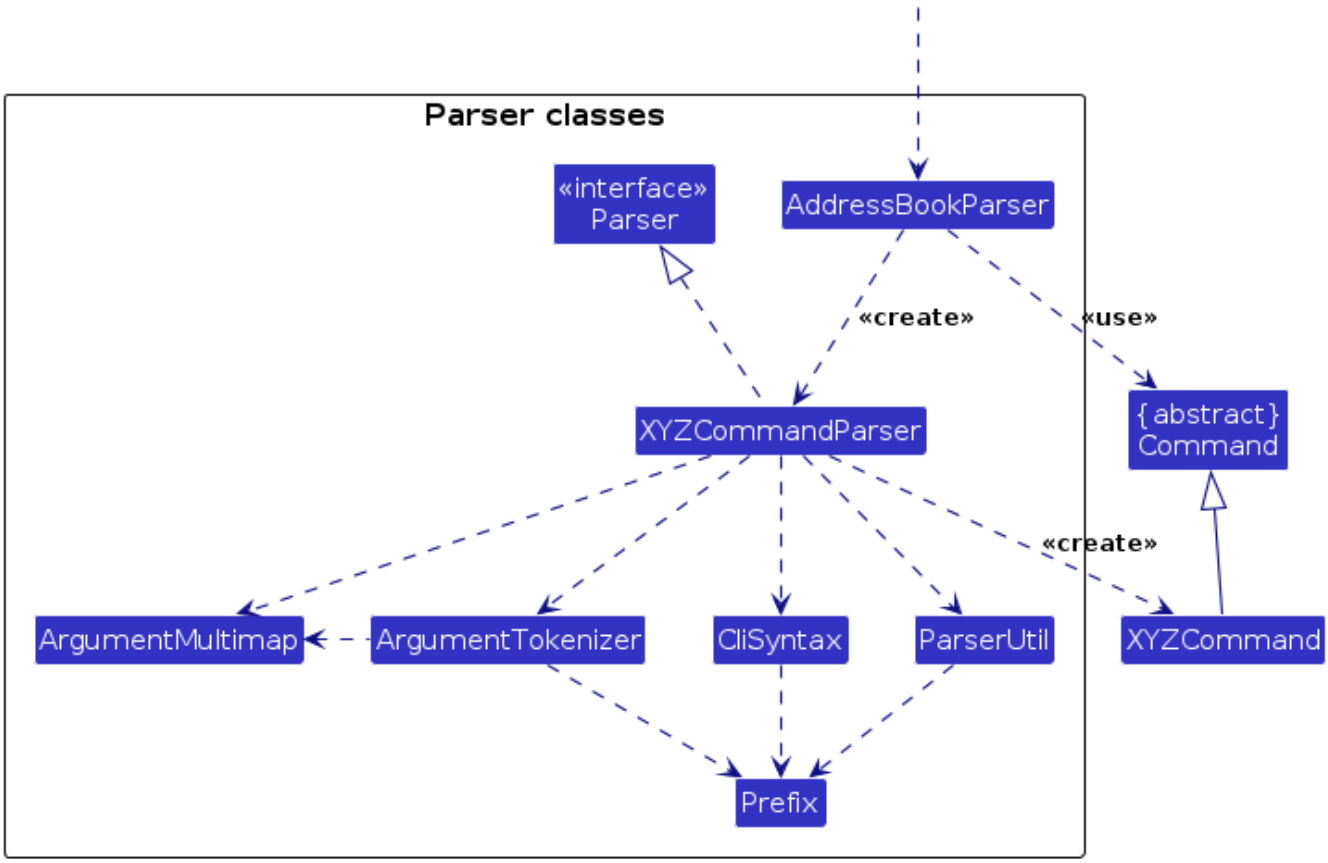


> **Note:** The lifeline for `DeleteCommandParser` should end at the destroy marker (X) but due to a limitation of PlantUML, the lifeline continues till the end of diagram.

How the `Logic` component works:

1. When `Logic` is called upon to execute a command, it is passed to an `AddressBookParser` object which in turn creates a parser that matches the command (e.g., `DeleteCommandParser`) and uses it to parse the command.
2. This results in a `Command` object (more precisely, an object of one of its subclasses e.g., `SingleDeleteCommand`) which is executed by the `LogicManager`.
3. The command can communicate with the `Model` when it is executed (e.g. to delete a person). Note that although this is shown as a single step in the diagram above (for simplicity), in the code it can take several interactions (between the command object and the `Model`) to achieve.
4. The result of the command execution is encapsulated as a `CommandResult` object which is returned back from `Logic`.

Here are the other classes in `Logic` (omitted from the class diagram above) that are used for parsing a user command:

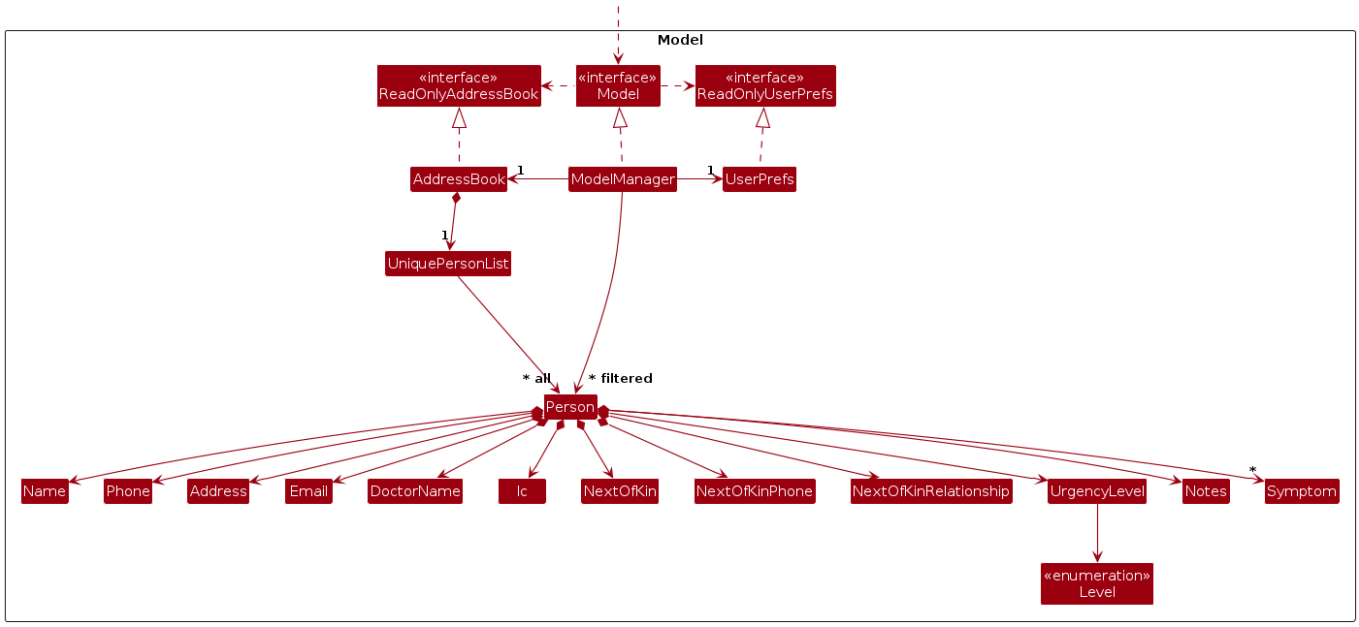


How the parsing works:

- When called upon to parse a user command, the `AddressBookParser` class creates an `XYZCommandParser` (`XYZ` is a placeholder for the specific command name e.g., `AddCommandParser`) which uses the other classes shown above to parse the user command and create a `XYZCommand` object (e.g., `AddCommand`) which the `AddressBookParser` returns back as a `Command` object.
- All `XYZCommandParser` classes (e.g., `AddCommandParser` , `DeleteCommandParser` , ...) inherit from the `Parser` interface so that they can be treated similarly where possible e.g, during testing.

Model component

API : `Model.java`



The `Model` component,

- stores the address book data i.e., all `Person` objects (which are contained in a `UniquePersonList` object).
- stores the currently 'selected' `Person` objects (e.g., results of a search query) as a separate *filtered* list which is exposed to outsiders as an unmodifiable `ObservableList<Person>` that can be 'observed' e.g. the UI can be bound to this list so that the UI automatically updates when the data in the list change.
- stores a `UserPref` object that represents the user's preferences. This is exposed to the outside as a `ReadOnlyUserPref` objects.
- does not depend on any of the other three components (as the `Model` represents data entities of the domain, they should make sense on their own without depending on other components)

In ClinicConnect, the Person model has been extended beyond the original AB3 design to support the polyclinic triage workflow. Besides standard contact details, each Person includes additional patient-related fields such as Ic, Doctor, UrgencyLevel, NextOfKin, NextOfKinPhone, Symptom, and Notes.

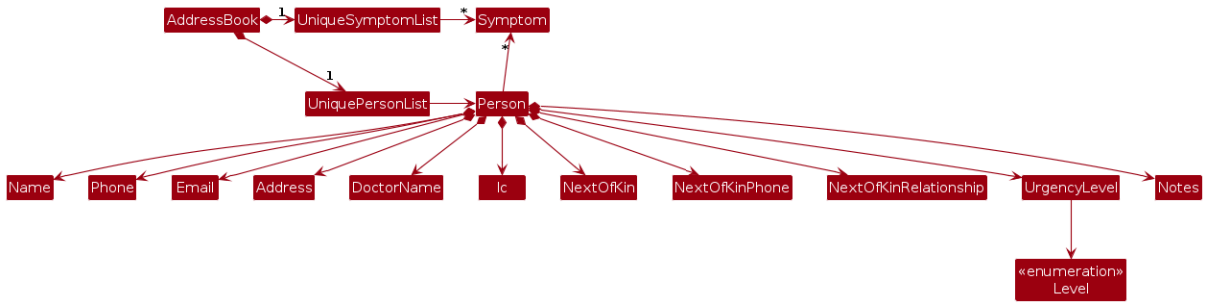
Some key ClinicConnect-specific model classes are:

- `Ic` : represents the patient's identification number. It is a core field used to uniquely identify patients and is used in duplicate detection.
- `UrgencyLevel` : represents the patient's triage priority. It is used to classify cases into levels such as low, moderate, high, and extreme, which supports prioritization during patient coordination.
- `Symptom` : represents a symptom tagged to a patient. A patient may have multiple symptoms recorded to support filtering and viewing of medical conditions.
- `Doctor` : represents the doctor currently assigned to the patient.
- `NextOfKin` , `NextOfKinRelationship` and `NextOfKinPhone` : represent the patient's emergency contact details. These classes capture the contact's name, relationship to the

patient, and phone number respectively, allowing the system to store structured next-of-kin information for coordination and emergency communication.
- `Notes` : stores additional remarks relevant to the patient's case.

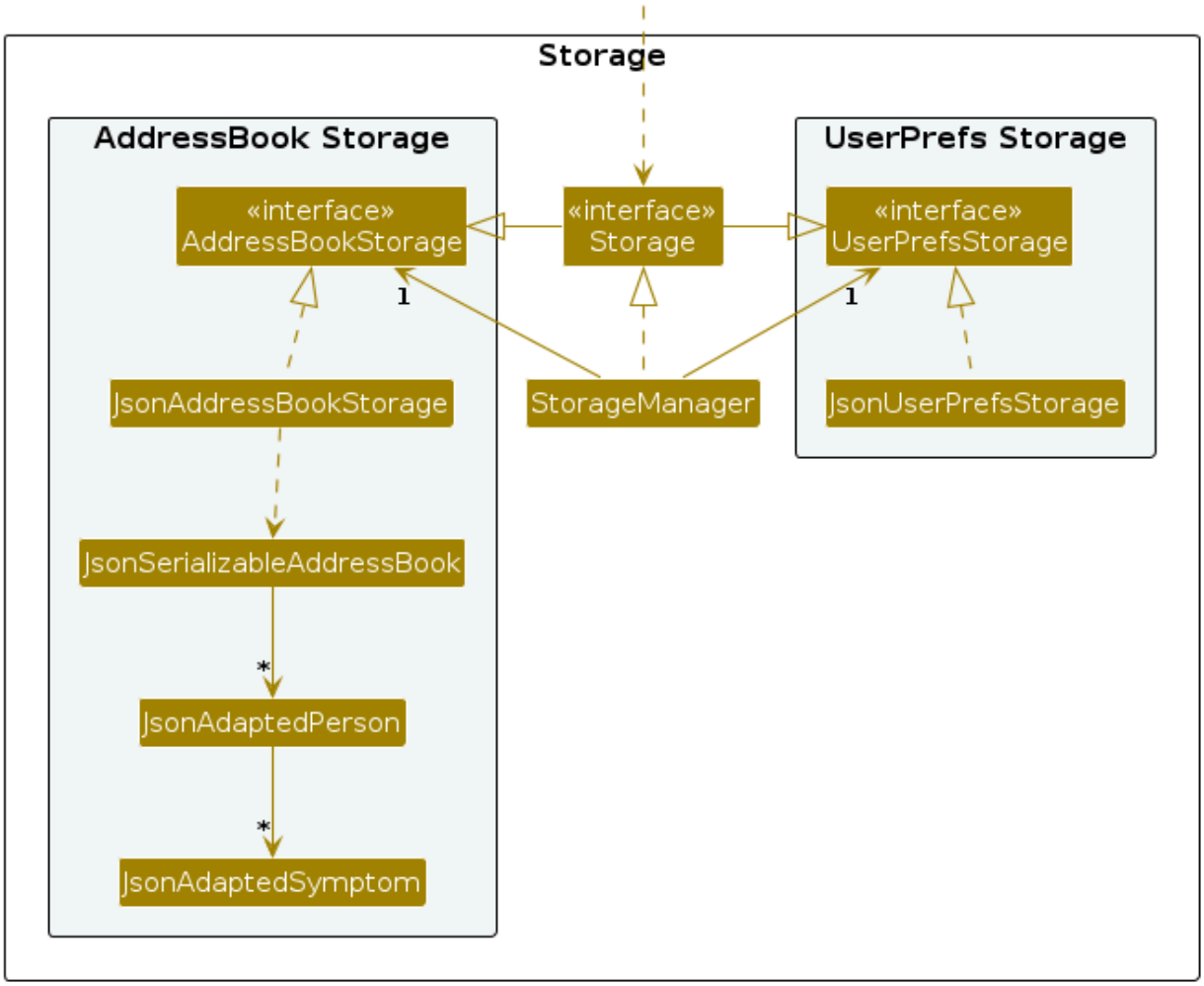
These domain classes are central to ClinicConnect because they adapt the original AB3 contact-management model into a patient coordination system suitable for triage operations in public polyclinics.

> **Note:** An alternative (arguably, a more OOP) model is given below. It has a `Symptom` list in the `AddressBook`, which `Person` references. This allows `AddressBook` to only require one `Symptom` object per unique symptom, instead of each `Person` needing their own `Symptom` objects.



Storage component

API : `Storage.java`



The `Storage` component,

- can save both address book data and user preference data in JSON format, and read them back into corresponding objects.
- inherits from both `AddressBookStorage` and `UserPrefStorage`, which means it can be treated as either one (if only the functionality of only one is needed).
- depends on some classes in the `Model` component (because the `Storage` component's job is to save/retrieve objects that belong to the `Model`)

Common classes

Classes used by multiple components are in the `seedu.address.commons` package.

Implementation

This section describes some noteworthy details on how certain features are implemented.

Automatic Triage Sorting

The Automatic Triage Sorting mechanism ensures that the patient list is always ordered by medical severity, allowing triage coordinators to identify critical cases immediately.

This feature is facilitated by the `Urgency` and `Ic` classes, and is integrated directly into the `Person` model. The sorting logic relies on Java's `Comparable<T>` interface to enforce a strict, deterministic order every time the list of patients is modified (e.g., during an `add` or `update` command).

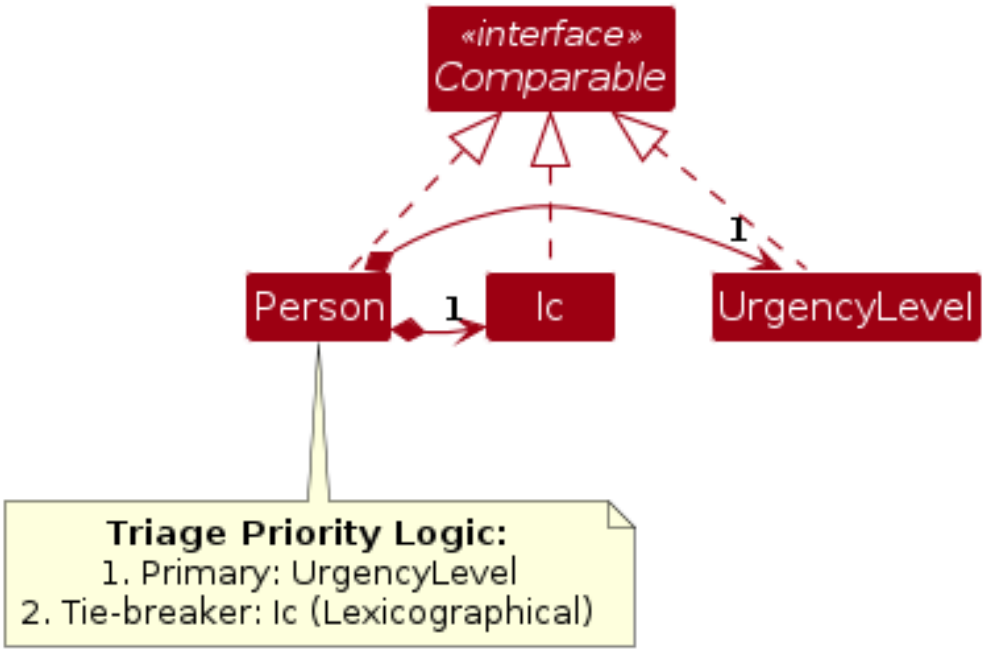
> **Note:** The sorting logic comparison can be conceptually represented by the following mathematical formula: `Triage Priority = (Urgency Weight * 10) + IC Lexicographical Value`

Sorting Logic Flow

1. **Primary Sort (Urgency):** The `Urgency` class implements `Comparable<Urgency>`. It assigns internal weightages to its valid states (`EXTREME` = 4, `HIGH` = 3, `MODERATE` = 2, `LOW` = 1). When two patients are compared, the one with the higher urgency weight is placed higher in the list.
2. **Secondary Sort (Tie-Breaker):** If two patients have the exact same `Urgency`, the sorting logic falls back to comparing their `Ic` values. To prevent mixed-case inputs (e.g., `s1234567a` vs `S1234567A`) from causing non-deterministic sorting in the UI, this tie-breaker uses **case-insensitive lexicographical string comparison** (via `String.CASE_INSENSITIVE_ORDER`).

Class Diagram

The following class diagram shows how the `Comparable` interface is implemented across the domain models:



Sequence of Events

When a user executes a command that alters the patient list (like `add` or `update`), the internal list must re-sort itself.

1. The `LogicManager` executes the command, which calls `Model#addPerson(Person)`.
2. The `ModelManager` adds the patient to the `UniquePersonList`.
3. The `UniquePersonList` internal list (or a wrapping `SortedList` in the UI) triggers the `Person#compareTo(Person)` method.
4. The list is re-ordered and the UI updates automatically to reflect the new triage hierarchy.

Design Considerations: Triage Sorting

Aspect: How to maintain the sorted order of the patient list.

- **Alternative 1 (Current Implementation):** Implement `Comparable` in `Model` and enforce sorting at the `UniquePersonList` level or UI `SortedList` level automatically upon any modification.
  - *Pros:* The UI and internal data are always inherently synchronized. No explicit `sort` command needs to be called by the user, fitting the fast-paced triage environment.
  - *Cons:* Slight performance overhead since sorting occurs on every single addition or update (O(n log n) time complexity per modification).
- **Alternative 2:** Do not sort automatically. Create a separate `sort` command that the user must type manually.
  - *Pros:* Better performance during rapid, sequential `add` operations.
  - *Cons:* Highly dangerous in a medical triage context. If a user forgets to type `sort`, an `EXTREME` urgency patient might be left at the bottom of the list, leading to critical delays in care.

Reason for choosing Alternative 1: In a clinical setting, data accuracy and immediate visibility of critical patients strictly outweigh the minor performance cost of automatic sorting. "Security by Design" principles dictate that the system should fail-safe; automatically sorting ensures no critical patient is ever accidentally hidden.

List feature

Implementation

The list mechanism lets users view either **all** patients or a **filtered** subset by **urgency level** (`u/`) and/or **symptoms** (`s/`). It is facilitated by **`ListCommand`** and **`ListCommandParser`**, which implement the following behaviour:

- **`ListCommandParser#parse(String)`** — Tokenizes `u/` and `s/` prefixes via `ArgumentTokenizer`, validates urgency values with `ParserUtil#parseUrgencyLevel` and symptom names with `ParserUtil#parseSymptom`, and builds a **`Predicate<Person>`**. If the user leaves arguments empty, it returns a `ListCommand` that uses `Model#PREDICATE_SHOW_ALL_PERSONS`, mimicking the behaviour of the original List feature.
- **`ListCommand#execute(Model)`** — Calls **`Model#updateFilteredPersonList(predicate)`**, then formats feedback from the size of **`Model#getFilteredPersonList()`** (success with count, empty-book message, or a no-matches message when filters return zero results).

Unprefixed text after `list` (e.g. `list fever`) is **rejected** so that filtering stays explicit and consistent with the rest of the CLI.

Combining filters: When both urgency and symptom criteria are present, a person matches if they satisfy **either** dimension (logical **OR**). Multiple `u/` or `s/` repetitions are normalized to sets; within each set, a match on **any** listed urgency (respectively symptom) is enough.

Data and undo: Listing does **not** change stored patient data; it only updates the **filtered view** in the model. `ListCommand` does not override `Command#isUndoable()`, so it remains **non-undoable**, like `find`.

Given below is a example usage scenario.

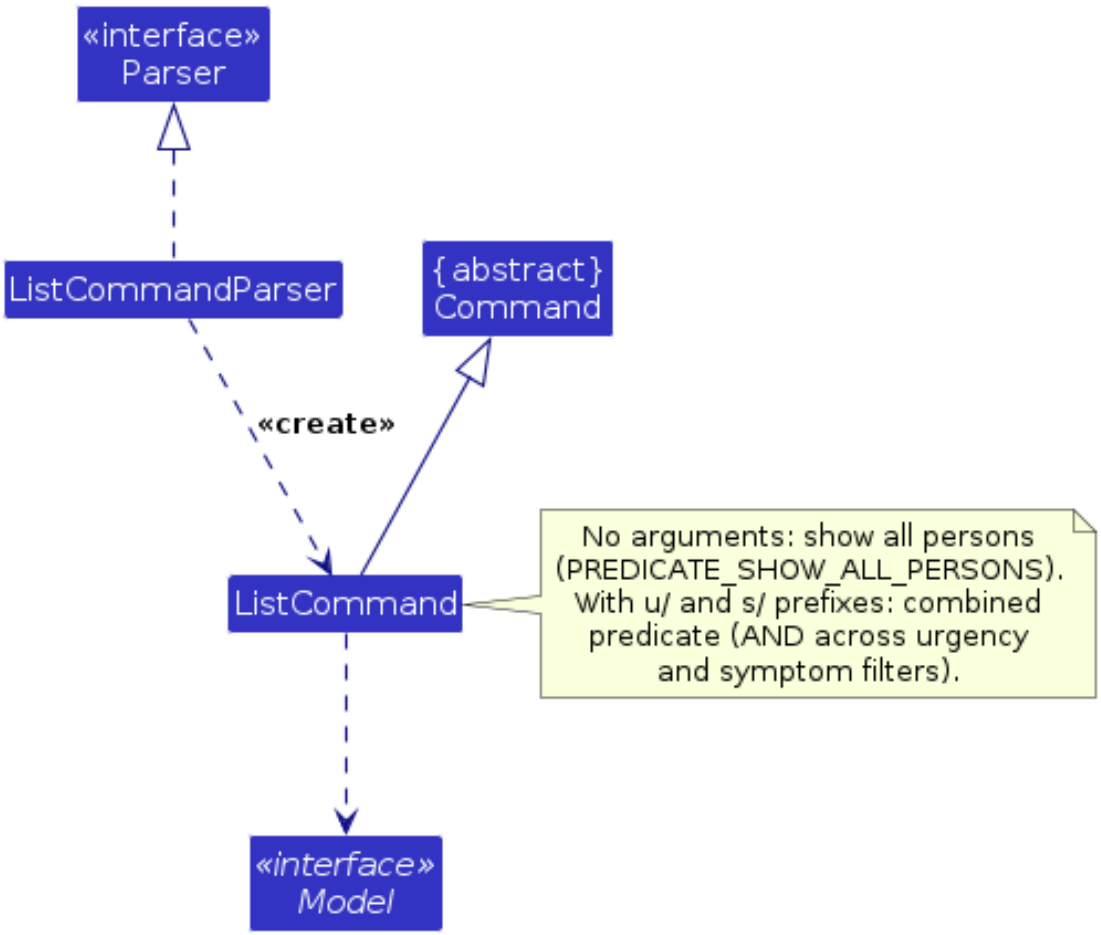
Step 1. The user previously ran a command that narrowed the UI (for example `find`), and wants to see every patient again. They execute `list` with no parameters. `ListCommandParser` returns a `ListCommand` with `PREDICATE_SHOW_ALL_PERSONS`; after execution, the UI shows the full list.

Step 2. The user executes `list u/high` to focus on high-urgency patients. The parser builds a predicate that keeps only persons whose urgency string matches `high`; `Model#updateFilteredPersonList` updates the `FilteredList`.

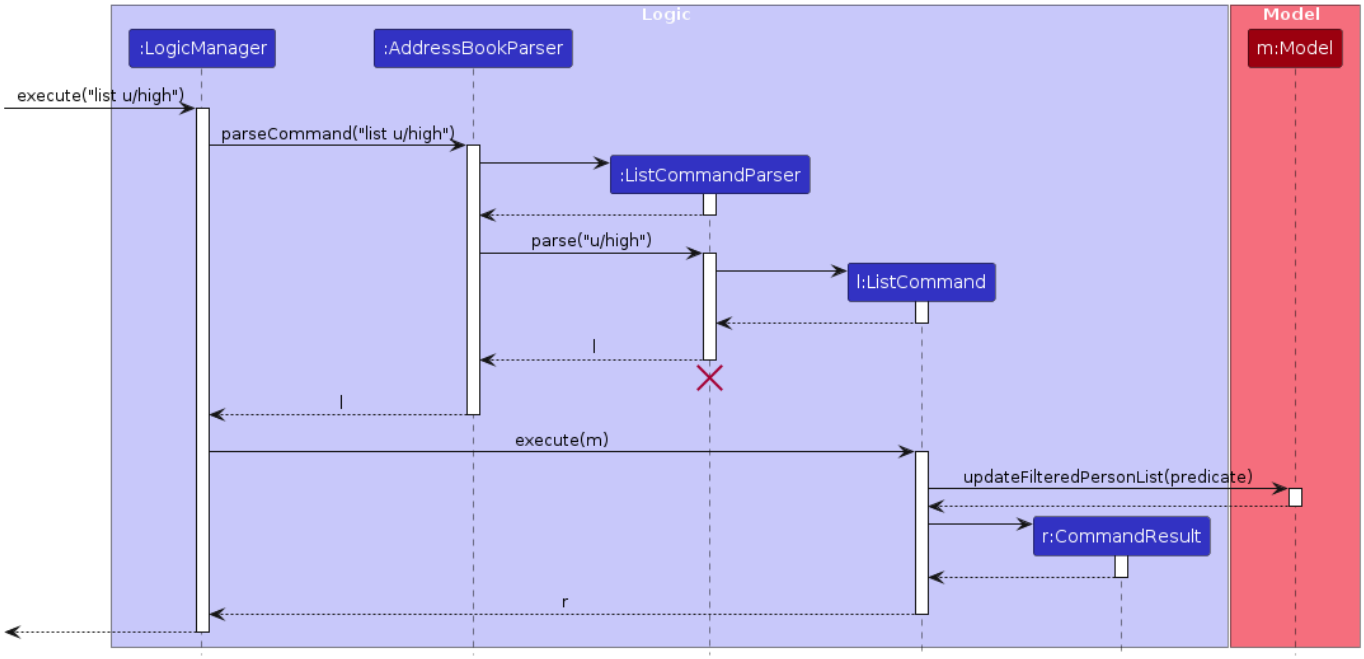
Step 3. The user executes `list u/high s/fever`. The parser builds a predicate so that persons who are **either** high urgency **or** have a fever symptom (case-insensitive) remain visible.

> **Note:** If the address book is empty, `list` with no filter returns an empty-list message. If filters are provided but **no** person matches, `ListCommand` returns a normal success message indicating no matches for the provided criteria.

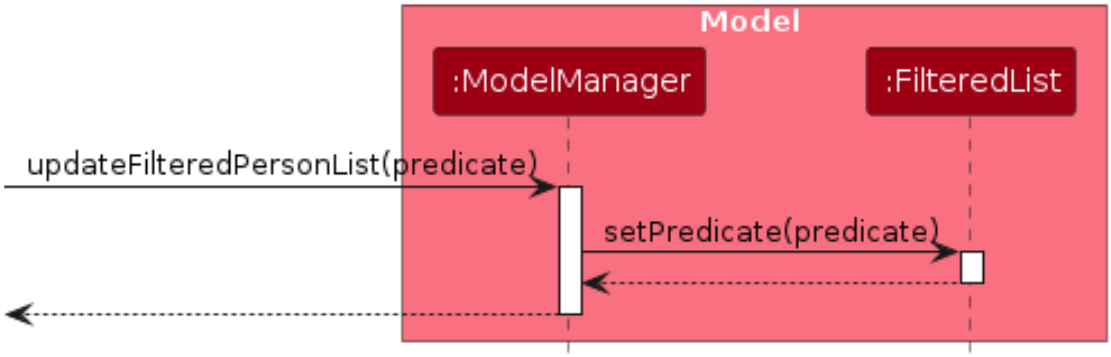
The class diagram below summarizes the main types involved in the list feature:



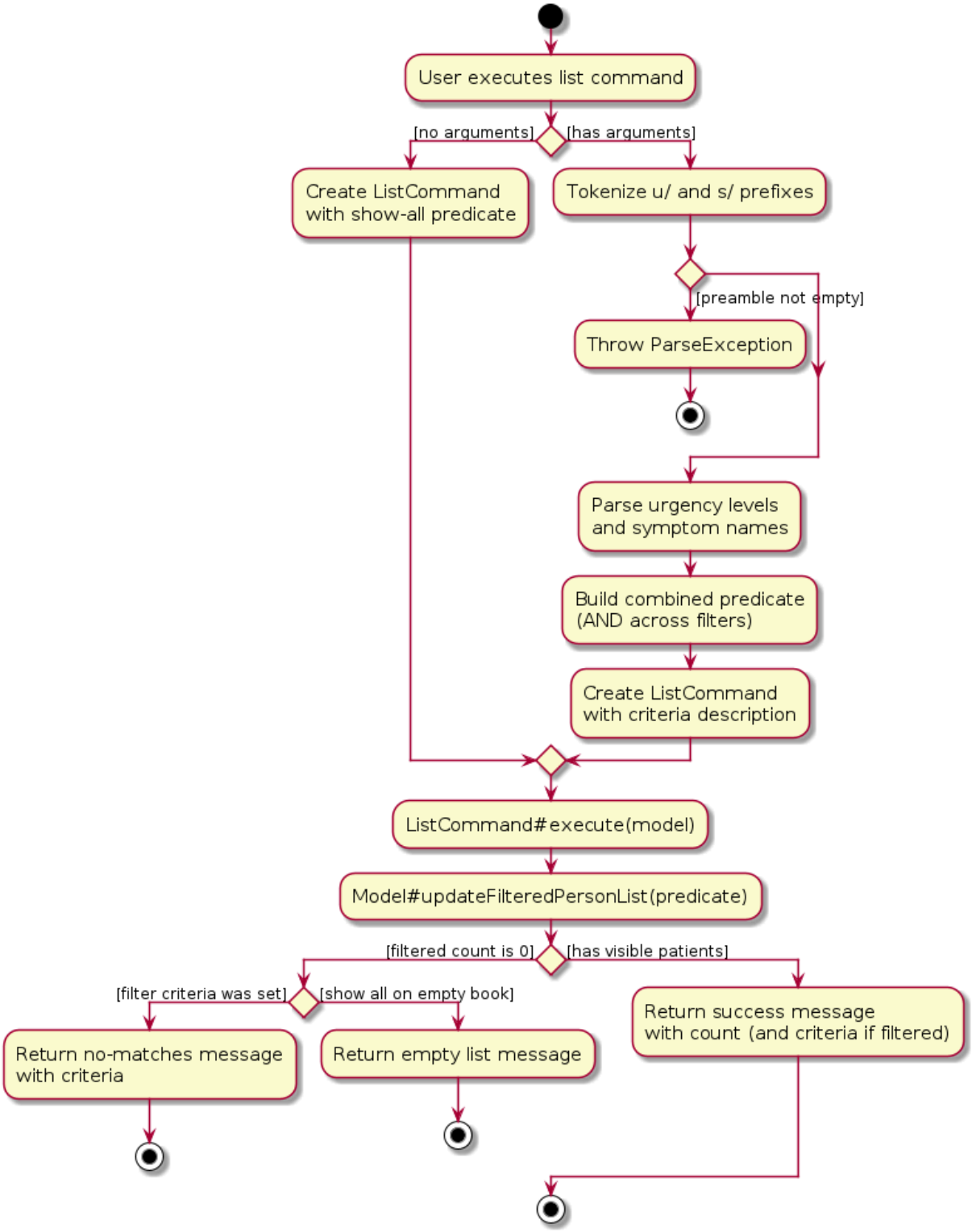
The following sequence diagram shows how a filtered `list` command flows through the **Logic** component (example: `list u/high`):



How `Model#updateFilteredPersonList` updates the view is shown below (concrete class `ModelManager` delegates to JavaFX `FilteredList`):



The activity diagram below summarizes parsing and execution outcomes:



Design considerations

Aspect: Strict prefixes vs free-text filtering

- **Alternative 1 (current choice):** Require `u/` and `s/`; reject unprefixed arguments.
  - *Pros:* Unambiguous parsing; aligns with `add` / `update` style; fewer accidental partial matches.
  - *Cons:* Slightly more typing than a single free-text query.
- **Alternative 2:** Accept raw keywords (e.g. `list fever`) and guess intent.
  - *Pros:* Faster for casual typing.
  - *Cons:* Ambiguity between urgency labels, symptom names, and future fields; worse error messages.

Find feature

Implementation

The find mechanism lets users search for patients using prefixed identifiers: patient name (`pn/`), IC (`ic/`), phone (`p/`), email (`e/`), and doctor (`d/`). It is facilitated by **`FindCommand`** and **`FindCommandParser`**, which implement the following behaviour:

- **`FindCommandParser#parse(String)`** — Tokenizes supported prefixes with `ArgumentTokenizer`, rejects unknown prefixes, rejects unprefixed preamble text, validates each provided value via `ParserUtil`/model constraints, and combines all provided criteria into a **single OR predicate**.
- **`FindCommand#execute(Model)`** — Calls **`Model#updateFilteredPersonList(predicate)`** and returns a result message containing the number of matched patients and a description of the criteria used.

Matching semantics: Search is case-insensitive. Patients are returned if they match **at least one** provided prefixed criterion (logical **OR** across fields). For keyword-based fields (`pn/`, `e/`, `d/`), matching uses token containment rules implemented by the corresponding predicate classes.

Input strictness: At least one supported prefix is required. Unknown prefixes, unprefixed preamble text, and duplicate occurrences of the same prefix are rejected to keep command interpretation unambiguous.

Data and undo: Finding does **not** modify stored patient data; it only updates the **filtered view** in the model. `FindCommand` remains **non-undoable**.

Given below is a example usage scenario.

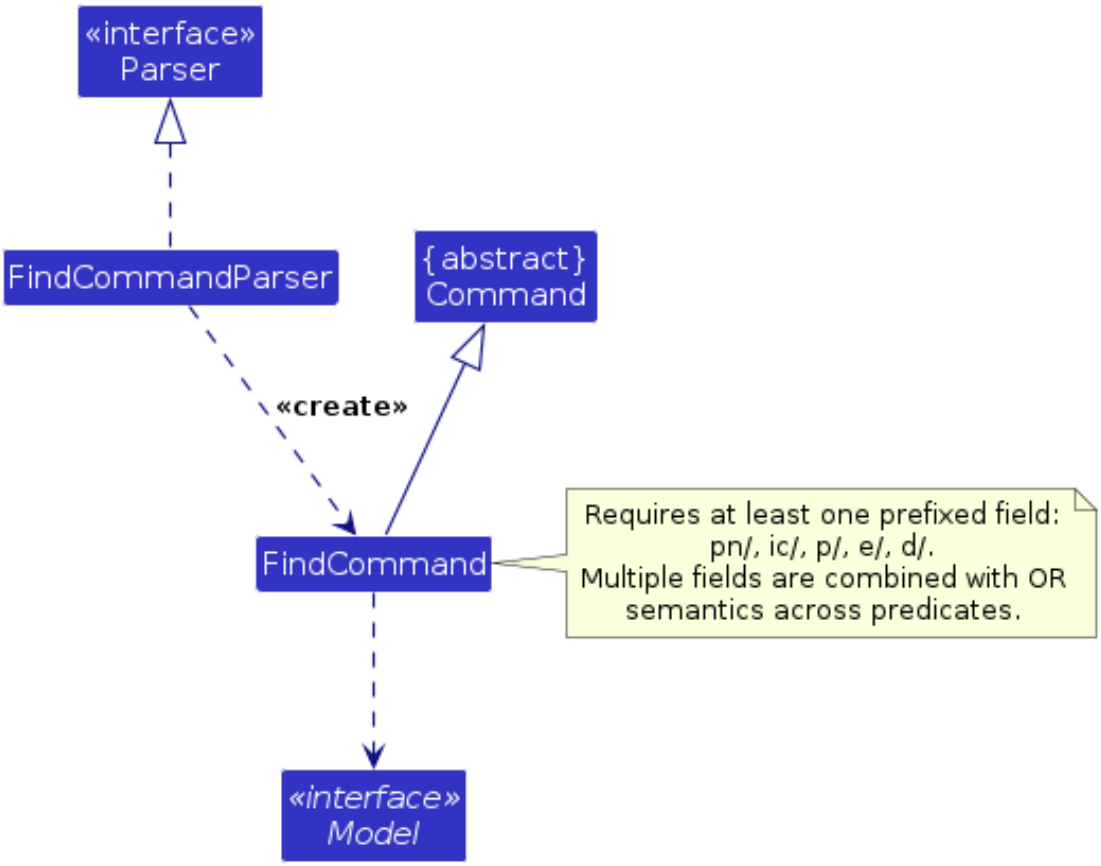
Step 1. The user executes `find pn/Alice`. The parser validates the name token(s), creates a `NameContainsKeywordsPredicate`, and `FindCommand` updates the filtered list to matching patients.

Step 2. The user executes `find ic/S1234567A e/alice@example.com`. The parser validates both fields and combines predicates using OR semantics. Patients matching either IC or email are shown.

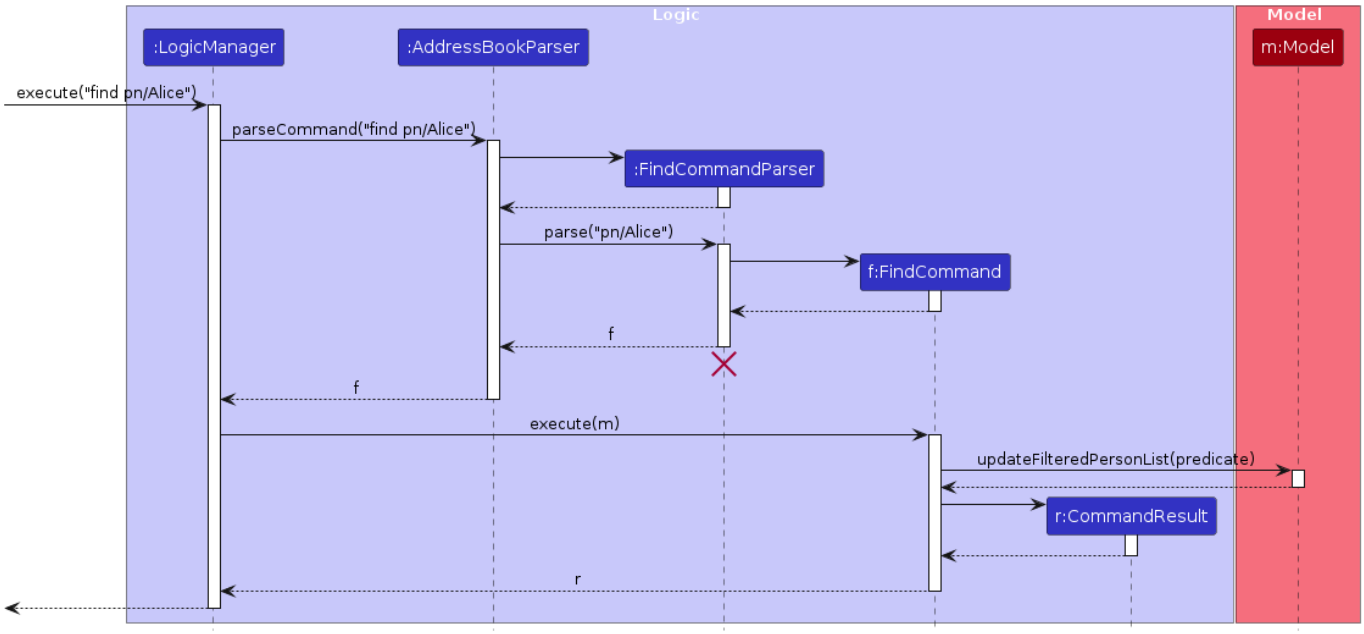
Step 3. The user executes `find x/value`. The parser detects an unsupported prefix and rejects the command with a parse error message indicating allowed prefixes.

> **Note:** `find` never throws a no-matches execution error. If no patient matches the provided criteria, the command still succeeds and reports `Found 0 patient(s) ...`.

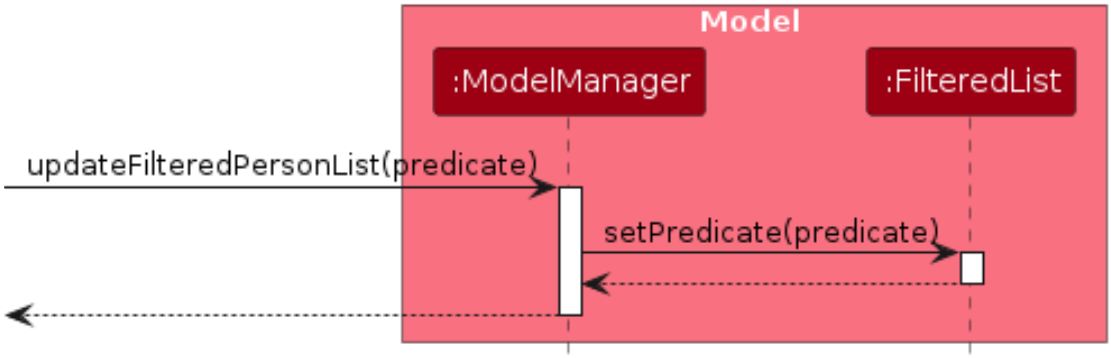
The class diagram below summarizes the main types involved in the find feature:



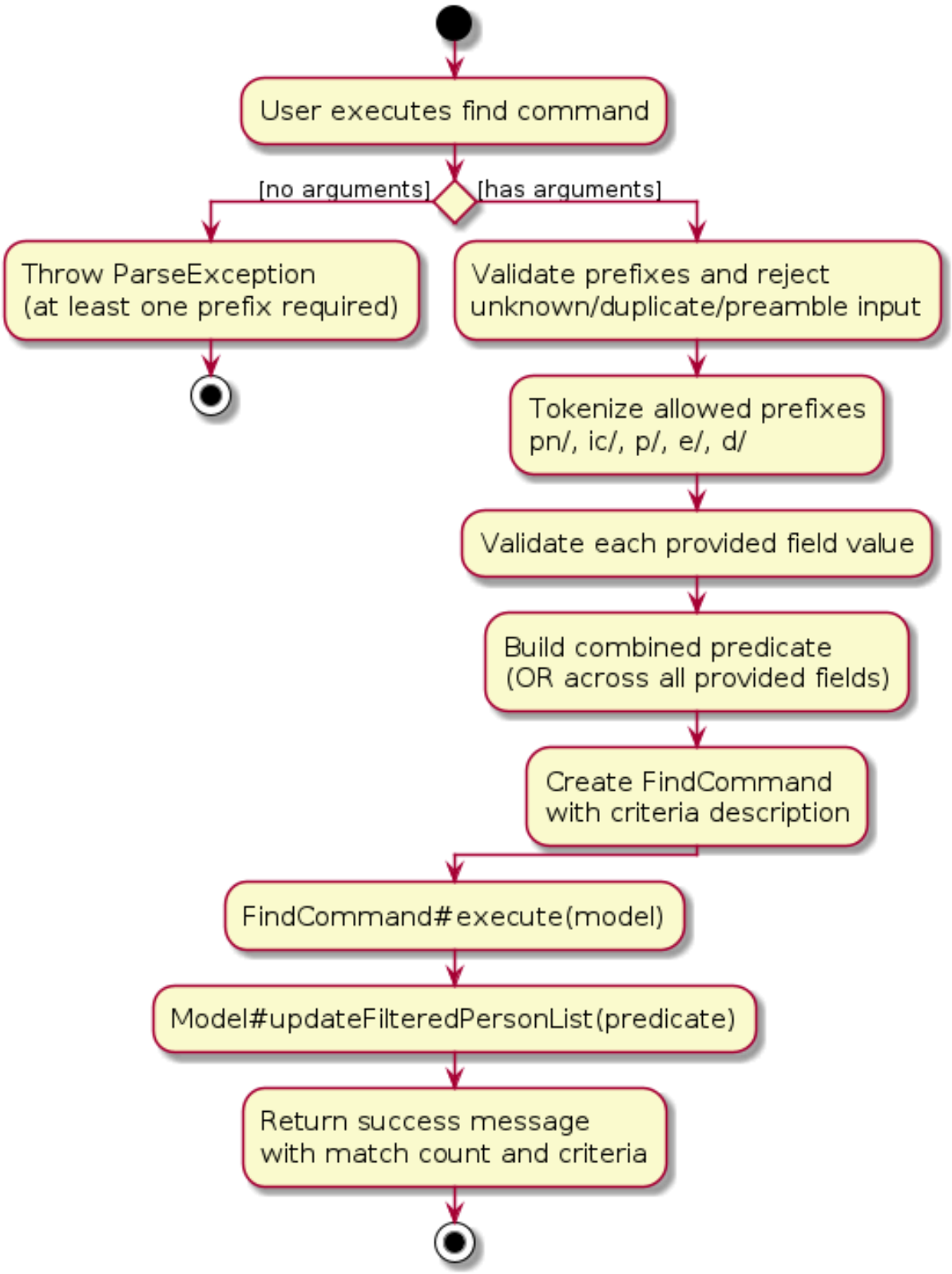
The following sequence diagram shows how a prefixed `find` command flows through the **Logic** component (example: `find pn/Alice`):



How `Model#updateFilteredPersonList` updates the view for `find` is shown below (concrete class `ModelManager` delegates to JavaFX `FilteredList`):



The activity diagram below summarizes parsing and execution outcomes:



Design considerations

Aspect: Strict prefixed input vs permissive free-text search

- **Alternative 1 (current choice):** Require explicit prefixes (`pn/` , `ic/` , `p/` , `e/` , `d/`) and reject unknown/unprefixed input.
  - *Pros:* Unambiguous parsing, clearer error messages, and consistent command style with other prefixed commands.
  - *Cons:* Slightly more typing for users who only want a quick name search.
- **Alternative 2:** Allow free-text terms (e.g. `find alice`) and infer the intended field.

- *Pros:* Faster input for simple searches.
- *Cons:* Ambiguous interpretation across fields and weaker validation feedback.

Undo Feature

The undo feature allows users to reverse the most recent data-modifying command. This is facilitated through the `Command` interface's `undo()` method and `isUndoable()` check.

Design

How Undo Works:

1. All commands inherit from the abstract `Command` class
2. Data-modifying commands (e.g., `AddCommand`, `DeleteCommand`, `UpdateCommand`, `ClearCommand`) override the `isUndoable()` method to return `true`
3. When a data-modifying command is executed, it stores necessary data to restore the previous state using fields like `wasExecuted` and backup data (e.g., `deletedPerson`, `clearedAddressBook`)
4. The `UndoCommand` maintains a reference to the most recent command via `setLastCommand()`
5. When `undo` is executed, it calls the `undo()` method on the stored command to restore the previous state

Undoable Commands

The following commands support undo:

- **`AddCommand`**: Stores the added person and removes it on undo
- **`SingleDeleteCommand`**, **`MultipleDeleteCommand`**, **`RangeDeleteCommand`**: Store deleted persons and re-add them on undo. Also support field deletion (symptoms, notes)
- **`SingleUpdateCommand`**: Stores the original person and restores it on undo. Note: `MultipleUpdateCommand` does not support undo
- **`ClearCommand`**: Stores a backup of the entire address book and restores it on undo

Implementation Details

Command Interface:

```
public abstract class Command {
    public boolean isUndoable() {
        return false; // Default: not undoable
    }

    public void undo(Model model) throws CommandException {
        // Default: no-op
    }
}
```

Example: AddCommand Undo Implementation

```java
public class AddCommand extends Command {
    private final Person toAdd;
    private boolean wasExecuted = false;

    @Override
    public boolean isUndoable() {
        return true;
    }

    @Override
    public CommandResult execute(Model model) {
        model.addPerson(toAdd);
        wasExecuted = true;
        return new CommandResult("Person added");
    }

    @Override
    public void undo(Model model) throws CommandException {
        if (wasExecuted) {
            model.deletePerson(toAdd);
        }
    }
}
```

Undo Flow

1. User enters a data-modifying command (e.g., `add pn/John ...`)
2. `LogicManager` executes the command via `execute(model)`
3. The command modifies the model and stores state for undo
4. User enters `undo`
5. `UndoCommand` calls `lastCommand.undo(model)` to restore the previous state
6. If no previous command exists or the last command was read-only, `UndoCommand` throws an error

Limitations

- **Single Undo Only**: Only the most recent command can be undone. Users cannot undo multiple commands in sequence
- **No Redo**: There is no redo functionality to reverse an undo operation
- **Read-Only Commands Ignored**: Commands like `list`, `find`, `help`, and `exit` do not modify data and cannot be undone
- **Bulk Update Not Supported**: `MultipleUpdateCommand` (bulk updates) does not support undo, only `SingleUpdateCommand` does

Future Enhancements

Potential improvements to the undo feature:

- Implement command history to support multiple levels of undo/redo
- Add support for bulk update undo in `MultipleUpdateCommand`

- Implement selective field undo for update commands

Command History Navigation

Implementation

The command history navigation feature allows users to cycle through previously successfully executed commands using the up/down arrow keys. The command history is kept so that there are no duplicated commands. This is implemented in the `CommandHistory` class, which maintains a list of previously executed commands strings that are trimmed to remove leading and trailing whitespaces, a pointer to the current position in that history, and the current user input being typed. The command history is session-based, meaning that the command history is cleared when the app is closed and does not persist across sessions.

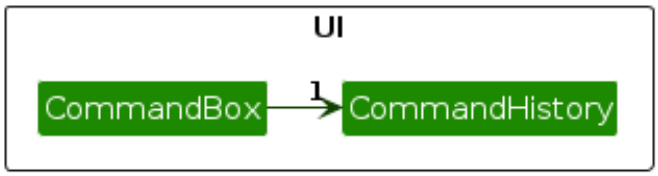
The `CommandHistory` class provides the following methods:

- `CommandHistory#add(String command)` — Adds a new command to the history and resets the pointer to the end of the list.
- `CommandHistory#getPreviousCommand(String currentInput)` — Moves the pointer up to the previous command and returns it. If the pointer is already at the top of the history, it remains there and returns the same command.
- `CommandHistory#getNextCommand()` — Moves the pointer down to the next command and returns it. If the pointer is already at the end of the history, it remains there and returns an empty string (indicating no command).
- `CommandHistory#removeAnyDuplicateCommands(String command)` — Removes any existing occurrence of the given command from the history to prevent duplicates.

> i
>
> - If there is a duplicate command in the history, it is removed before adding the new command. This ensures that the history contains only unique commands, and the most recent occurrence of a command is what gets stored.
>
> - Duplicate commands are identified using case-insensitive comparison after trimming leading and trailing whitespace. A command is considered a duplicate only if it **exactly** matches an existing command in the history list.

The `CommandBox` class listens for key presses and interacts with the `CommandHistory` to update the command input field accordingly when the user presses the up/down arrow keys.

Here is a class diagram summarizing the main types involved in the command history feature:



Given below is an example usage scenario. Assume there are already some patient records in the address book before the user executes any command.

Step 1. The user executes the command `add pn/Alice …`. The `CommandHistory` adds this command to its history list and resets the pointer to the end.

Step 2. The user executes the command `delete 1`. The `CommandHistory` adds this command to its history list and resets the pointer to the end.

Step 3. The user enters a partial command `add pn/Bob` but has not executed it yet. This partial input is stored in the `CommandHistory` as the current user input, but it is not added to the history list until it is executed.

Step 4. The user now presses the up arrow key. The `CommandHistory` moves the pointer up to the previous command (`delete 1`) and returns it, which is then displayed in the command input field.

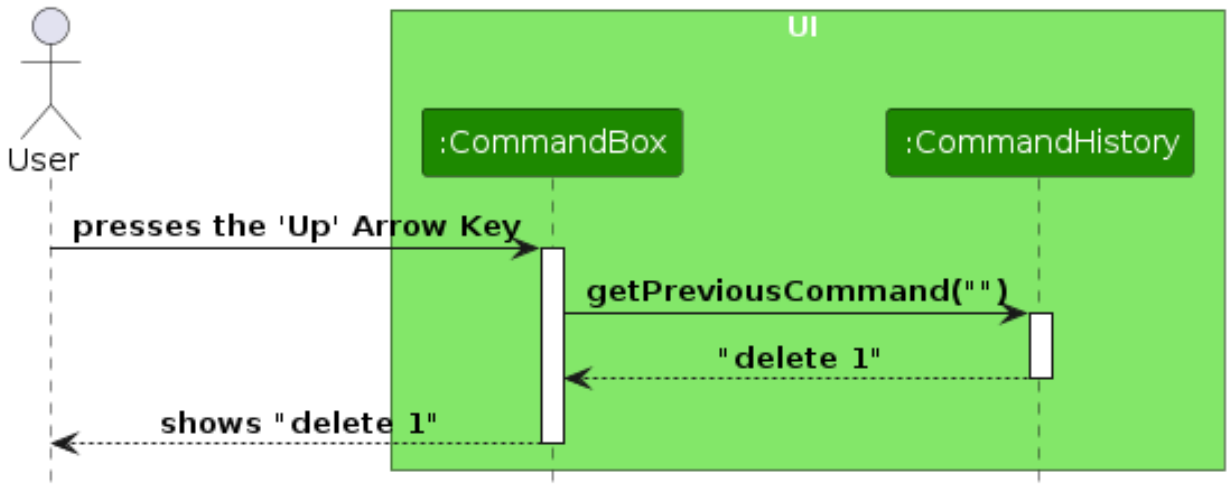
Step 5. The user presses the up arrow key again. The `CommandHistory` moves the pointer up to the previous command (`add pn/Alice …`) and returns it, which is then displayed in the command input field.

Step 6. The user presses the up arrow key again. Since the pointer is already at the top of the history, it remains there and returns the same command (`add pn/Alice …`), which is displayed in the command input field.

Step 7. The user now presses the down arrow key. The `CommandHistory` moves the pointer down to the next command (`delete 1`) and returns it, which is displayed in the command input field.

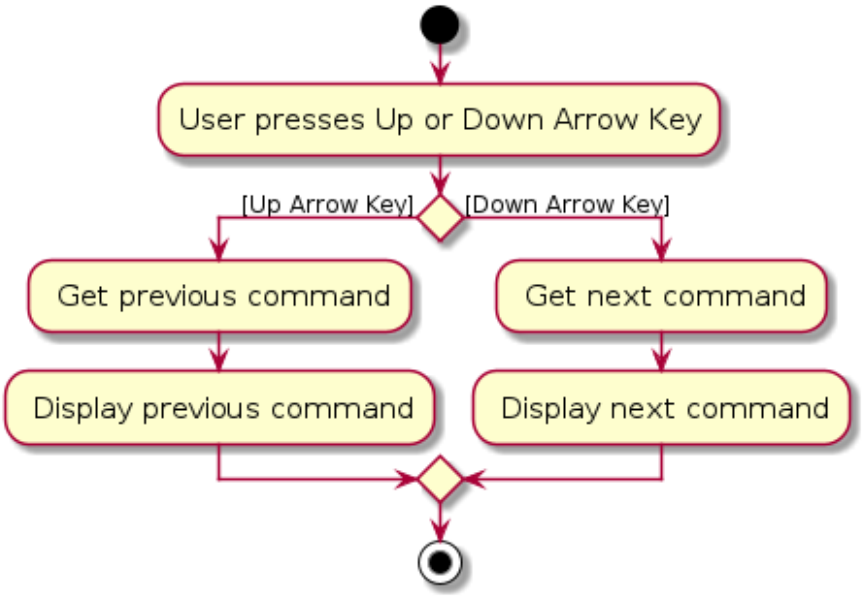
Step 8. The user presses the down arrow key again. The `CommandHistory` moves the pointer down to the end of the history (since there are no more commands), and returns the initial partial command input (`add pn/Bob`), which is displayed in the command input field.

Here is a sequence diagram showing how the command history navigation works when the user presses the up arrow key (Assuming there are already some commands in the history and that the previous command in the history is `delete 1`):

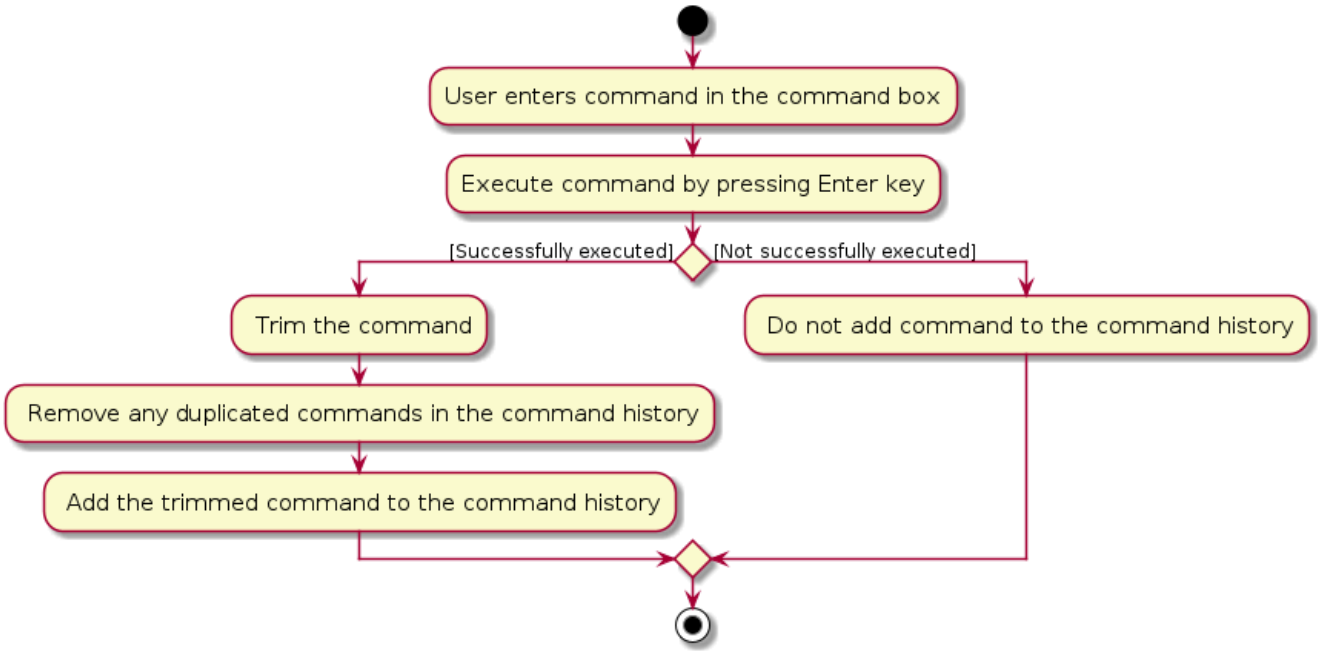


> Note that `getPreviousCommand("")` method is taking in an empty string, that means that the user has not typed anything in the command box before pressing the up arrow key.

The following activity diagram summarizes the behavior of the command history navigation when the user presses the up/down arrow keys:



The following activity diagram summarizes the behavior of the command history navigation when the user enters a new command (e.g., `add pn/Charlie …`) and executes it:



Documentation, logging, testing, configuration, dev-ops

- Documentation guide

- Testing guide
- Logging guide
- Configuration guide
- DevOps guide

Appendix: Requirements

Product scope

Target user profile:

- has a need to manage a significant number of patient records
- prefer desktop apps over other types
- can type fast
- prefers typing to mouse interactions
- is reasonably comfortable using CLI apps

Value proposition: This product eliminates the administrative friction of navigating complex, full-scale medical record systems during time-sensitive triage. By providing a streamlined, high-speed CLI interface for tracking patient updates and doctor assignments, this reduces coordinators' cognitive load, ensuring faster patient throughput and more accurate prioritisation in fast-paced, high-workload clinical environments.

User stories

Priorities: High (must have) - `* * *`, Medium (nice to have) - `* *`, Low (unlikely to have) - `*`

Priority	As a/an ...	I want to ...	So that I can...
`* * *`	beginner user	list all patients in the system	verify that the data I just entered was saved correctly.
`* * *`	beginner user	exit the app safely	be assured that the data entered into the app is saved when I end my shift.
`* * *`	user	add patient details	start tracking those patients.
`* * *`	triage coordinator	update specific details of a patient (such as their urgency level, symptoms, or notes)	quickly keep records current as the patient moves through the triage process and their medical condition changes.
`* * *`	user	remove patients	remove duplicates or wrongly created records to keep it accurate.

Priority	As a/an ...	I want to ...	So that I can...
* *	triage coordinator	search for patient records using different criteria (like Name, IC, Phone, or Urgency Level)	easily locate a specific patient in an emergency or find specific groups of patients without manually scrolling through the entire database.
* *	expert user	use my up/down keyboard keys	conveniently access previously entered commands in my history to execute again.
* *	expert user	archive old patient records	focus on the relevant patient records.
* *	beginner user	see a confirmation message after adding a patient	be assured that a patient has been added to my list .
* *	beginner user	use a help command	easily understand the capabilities of ClinicConnect.
* *	beginner user	reset the app to its initial state	safely experiment with the app.
* *	beginner user	see instructions to help me correct my mistakes	easily correct mistakes.
* *	user	sort patients based on their urgency level	quickly identify which patient requires immediate care.
* *	user	attach notes to patient records	capture contextual information.
* *	triage coordinator	perform batch edits or deletions	manage multiple patients efficiently.
* *	user	use shorter prefixes	get things done quickly.
* *	user	add an urgency level to each patient	know which patient requires immediate attention first.
* *	expert user	use a one-line command format (e.g., add n/ p/ ic/ cond/)	enter a patient's full clinical profile quickly in seconds.

Priority	As a/an ...	I want to ...	So that I can...
* *	expert user	filter patients using symptoms	categorise patients based on clinical priority or special needs.
* *	beginner user	undo a mistaken deletion from an entered command	recover accidentally deleted data without re-entering everything.
* *	beginner user	see the app populated with sample data	easily see how the app will look when it is in use.
* *	beginner user	search for a patient by their name, ic, doctor assigned and phone	quickly find an existing patient record.
* *	beginner user	undo recent actions	feel safe exploring the app.
* *	triage coordinator	perform batch edits or deletions	manage multiple patients efficiently
* *	user	filter patients by urgency level	prioritise higher urgency level
*	beginner user	skip onboarding steps	explore the application freely.
*	beginner user	view a brief onboarding walkthrough	know what the app can do.
*	beginner user	view what actions I have recently made	understand the current state of the database more accurately.
*	expert user	use command shortcuts or aliases when entering patient details	maximize my efficiency in typing during peak hours.
*	expert user	customize views or defaults	make the app matches my working habits.
*	user	manually save my data at any point	be assured that I saved my data.

Priority	As a/an ...	I want to ...	So that I can...
`*`	expert user	chain commands together in a single line (e.g., add ... && list)	execute multiple actions without waiting for intermediate prompts.

Use cases

(For all use cases below, the **System** is the `ClinicConnect` and the **Actor** is the `user`, unless specified otherwise)

Use case: UC1 - Add a patient record

Preconditions:

- The user has launched the app and is at the main window.

Guarantees:

- The patient will be added only if the user enters the format correctly, there is no duplicate patient record or there is no error with the save file.

MSS

1. The user requests to add a patient by providing their details.
2. ClinicConnect validates the input.
3. ClinicConnect adds the patient.
4. ClinicConnect saves the new data.
5. ClinicConnect displays a successful message.

   Use case ends.

Extensions

- 2a. ClinicConnect detects an error with the user's input.
  - 2a1. ClinicConnect displays an unsuccessful message.

    Use case resumes from step 1.
- 2b. ClinicConnect detects a duplicate patient record.

    - 2b1. ClinicConnect displays an unsuccessful message.

      Use case resumes from step 1.

- 3a. ClinicConnect detects an error with the save file.

    - 3a1. ClinicConnect displays an unsuccessful message.

      Use case ends. (Note: Since there is an issue with the save file, the use case should end here rather than retrying)

Use case: UC2 - Delete a patient record

Preconditions:

- The user has launched the app and is at the main window.

Guarantees:

- The patient will be deleted only if the user enters the format correctly, the list of patient records is not empty already or there is no error with the save file.

MSS

1. The user requests to delete a specific patient in the list.

2. ClinicConnect validates the input.

3. ClinicConnect deletes the patient.

4. ClinicConnect saves the new data.

5. ClinicConnect displays a successful message.

   Use case ends.

Extensions

- 2a. ClinicConnect detects an error with the user's input.

    - 2a1. ClinicConnect displays an unsuccessful message.

      Use case resumes from step 1.

- 2b. ClinicConnect detects that the list is already empty.

    - 2b1. ClinicConnect displays an unsuccessful message.

Use case ends. (Note: Since the list is empty, the use case should end here rather than retrying)

- 3a. ClinicConnect detects an error with the save file.
    - 3a1. ClinicConnect displays an unsuccessful message.

      Use case ends. (Note: Since there is an issue with the save file, the use case should end here rather than retrying)

Use case: UC3 - Delete a group of selected patient records

Preconditions:

- The user has launched the app and is at the main window.

Guarantees:

- The patients will be deleted only if the user enters the format correctly, the list of patient records is not empty already or there is no error with the save file.

MSS

1. The user requests to delete multiple specific patients in the list.
2. ClinicConnect validates the input.
3. ClinicConnect deletes the patients.
4. ClinicConnect saves the new data.
5. ClinicConnect displays a successful message.

   Use case ends.

Extensions

- 2a. ClinicConnect detects an error with the user's input.
    - 2a1. ClinicConnect displays an unsuccessful message.

      Use case resumes from step 1.
- 2b. ClinicConnect detects that the list is already empty.
    - 2b1. ClinicConnect displays an unsuccessful message.

Use case ends. (Note: Since the list is empty, the use case should end here rather than retrying)

- 3a. ClinicConnect detects an error with the save file.
    - 3a1. ClinicConnect displays an unsuccessful message.

      Use case ends. (Note: Since there is an issue with the save file, the use case should end here rather than retrying)

Use case: UC4 - Delete a range of selected patient records

Preconditions:

- The user has launched the app and is at the main window.

Guarantees:

- The patients will be deleted only if the user enters the format correctly, the list of patient records is not empty already or there is no error with the save file.

MSS

1. The user requests to delete a range of patients in the list.
2. ClinicConnect validates the input.
3. ClinicConnect deletes the patients.
4. ClinicConnect saves the new data.
5. ClinicConnect displays a successful message.

   Use case ends.

Extensions

- 2a. ClinicConnect detects an error with the user's input.
    - 2a1. ClinicConnect displays an unsuccessful message.

      Use case resumes from step 1.
- 2b. ClinicConnect detects that the list is already empty.
    - 2b1. ClinicConnect displays an unsuccessful message.

Use case ends. (Note: Since the list is empty, the use case should end here rather than retrying)

- 3a. ClinicConnect detects an error with the save file.
    - 3a1. ClinicConnect displays an unsuccessful message.

      Use case ends. (Note: Since there is an issue with the save file, the use case should end here rather than retrying)

Use case: UC5 - Update a patient's details

Preconditions:

- The user has launched the app and is at the main window.

Guarantees:

- The patient will be updated only if the user enters the format correctly, the list of patient records is not empty already or there is no error with the save file.

MSS

1. The user requests to update specific details of a patient in the list.
2. ClinicConnect validates the input.
3. ClinicConnect updates the patient's details.
4. ClinicConnect saves the new data.
5. ClinicConnect displays a successful message.

   Use case ends.

Extensions

- 2a. ClinicConnect detects an error with the user's input.
    - 2a1. ClinicConnect displays an unsuccessful message.

      Use case resumes from step 1.
- 2b. ClinicConnect detects that the list is empty.
    - 2b1. ClinicConnect displays an unsuccessful message.

Use case ends. (Note: Since the list is empty, the use case should end here rather than retrying)

- 3a. ClinicConnect detects an error with the save file.
    - 3a1. ClinicConnect displays an unsuccessful message.

      Use case ends. (Note: Since there is an issue with the save file, the use case should end here rather than retrying)

Use case: UC6 - Update multiple patients' details

Preconditions:

- The user has launched the app and is at the main window.

Guarantees:

- The patients will be updated only if the user enters the format correctly, the list of patient records is not empty, all provided indices are valid and unique, and there is no error with the save file.

MSS

1. The user requests to update specific details for multiple specific patients in the list.
2. ClinicConnect validates the input and ensures all indices are valid.
3. ClinicConnect updates the details of all specified patients.
4. ClinicConnect saves the new data.
5. ClinicConnect displays a successful message.

   Use case ends.

Extensions

- 2a. ClinicConnect detects an error with the user's input (e.g., invalid format, missing update fields, out-of-bounds indices, or duplicated indices).
    - 2a1. ClinicConnect displays an unsuccessful message. No patients are updated.

      Use case resumes from step 1.
- 2b. ClinicConnect detects that the list is empty.

    - 2b1. ClinicConnect displays an unsuccessful message.

      Use case ends. (Note: Since the list is empty, the use case should end here rather than retrying)

- 3a. ClinicConnect detects an error with the save file.

    - 3a1. ClinicConnect displays an unsuccessful message.

      Use case ends. (Note: Since there is an issue with the save file, the use case should end here rather than retrying)

Use case: UC7 - Search patient records

Preconditions:

- User has launched the app and is at the main window.

Guarantees:

- A list of patients matching the criteria will be displayed if the user enters the search format correctly with at least one valid identifier.

MSS

1. The user requests to find patients using specific search criteria.

2. ClinicConnect validates the input format.

3. ClinicConnect filters the current list of patients based on the criteria.

4. ClinicConnect updates the UI to display the matching patient records.

5. ClinicConnect displays a message showing the number of patients found.

   Use case ends.

Extensions

- 2a. ClinicConnect detects an error with the user's input (e.g., no fields provided).
    - 2a1. ClinicConnects display an unsuccessful message detailing the error.

      Use case resumes from step 1.

Non-Functional Requirements

1. Should work on any *mainstream OS* as long as it has Java `17` or above installed.

2. Should be able to hold up to 1000 persons without a noticeable sluggishness in performance for typical usage.
3. A user with above average typing speed for regular English text (i.e. not code, not system admin commands) should be able to accomplish most of the tasks faster using commands than using the mouse.
4. Should be fully operable using only the keyboard, with no mouse interaction required.
5. Should gracefully handle a corrupted or missing data file by starting with an empty address book, without crashing.
6. Should not lose any patient data if the application is closed unexpectedly (e.g. power failure), as data is saved after every command.

Glossary

- **Mainstream OS**: Windows, Linux, MacOS
- **Above average typing speed**: 40 words per minute (wpm) or more, which is the average typing speed for regular English text.
- **Patient record**: A record containing the details of a patient.
- **Patient details**: The details of a patient, which include their name, phone number, address, email, IC number, attending doctor, urgency level, symptoms and notes.
- **Triage coordinator**: A user who is responsible for managing the patient records and ensuring that patients are prioritized based on their urgency level.
- **Expert user**: A user who is familiar with the app and its features, and can use it efficiently to manage patient records.
- **Beginner user**: A user who is new to the app and may not be familiar with all its features, but can still use it to manage patient records with some guidance.
- **User**: A user, which can be either a triage coordinator, an expert user or a beginner user.
- **Address book**: The collection of patient records stored in the app, represented as an `AddressBook` object.
- **Person/Patient**: A patient record in the address book, represented as a `Person` object. They are used interchangeably, but they refer to the same thing in the context of this app.
- **Urgency**: A classification of a patient's medical severity. Valid levels are `EXTREME`, `HIGH`, `MODERATE`, and `LOW`, listed from highest to lowest priority.
- **IC number**: A unique national identification number used to identify a patient (e.g. `T0123456B`). Used as the primary key to prevent duplicate patient records.
- **Triage priority**: The computed order in which patients are ranked in the list, determined first by urgency level and then by IC number lexicographically as a tie-breaker.
- **Filtered list**: The subset of patient records currently displayed in the UI after applying search or filter criteria. Does not affect the underlying stored data.
- **Session**: A single run of the application from launch to close. Command history is session-based and does not persist across sessions.
- **Next-of-kin**: An emergency contact associated with a patient record, including their name, phone number, and relationship to the patient.

Appendix: Instructions for manual testing

Given below are instructions to test the app manually.

> **Note:** These instructions only provide a starting point for testers to work on; testers are expected to do more *exploratory* testing.

Launch and shutdown

1. Initial launch

    1. Download the jar file and copy into an empty folder

    2. Double-click the jar file Expected: Shows the GUI with a set of sample patient records. The window size may not be optimum.

2. Saving window preferences

    1. Resize the window to an optimum size. Move the window to a different location. Close the window.

    2. Re-launch the app by double-clicking the jar file.
    Expected: The most recent window size and location is retained.

Adding a person

1. Adding a person with all fields

    1. Prerequisites: Address book has no duplicated patient record with the same IC number as the person being added.
    2. Test case: `add pn/John Doe Jun Kai ic/T0123456B p/12345678 a/21 Serangan Road e/john@doe.com u/high d/Dr Tan Ah Beng nk/Mary Doe nkp/87654321 nkr/Mother s/Diabetic n/Admitted at 12pm`
    Expected: A new patient record is added to the list with the specified details.
2. Adding a person with only the required fields

    1. Prerequisites: Address book has no duplicated patient record with the same IC number as the person being added.
    2. Test case: `add pn/John Doe Jun Kai ic/T0123456B p/12345678 a/21 Serangan Road e/john@doe.com u/high d/Dr Tan Ah Beng nk/Mary Doe nkp/87654321 nkr/Mother`
    Expected: A new patient record is added to the list with the specified details. Notes is left empty. No symptoms are displayed.
3. Adding a person with a missing required field to an empty address book

    1. Prerequisites: Address book has no duplicated patient record with the same IC number as the person being added.
    2. Test case: `add pn/John Doe Jun Kai p/12345678 a/21 Serangan Road e/john@doe.com u/high d/Dr Tan Ah Beng nk/Mary Doe nkp/87654321 nkr/Mother` (missing IC prefix and

the associated value)
Expected: No patient record is added to the list. An error message is shown.

4. Adding a person where a prefix is present but its associated value is missing

    1. Prerequisites: Address book has no duplicated patient record with the same IC number as the person being added.
    2. Test case: `add pn/John Doe Jun Kai ic/ p/12345678 a/21 Serangan Road e/john@doe.com u/high d/Dr Tan Ah Beng nk/Mary Doe nkp/87654321 nkr/Mother s/Diabetic n/Admitted at 12pm` (IC prefix is present but the associated value is missing)
    Expected: No patient record is added to the list. An error message is shown.

5. Adding a person with duplicated compulsory prefixes

    1. Prerequisites: Address book has no duplicated patient record with the same IC number as the person being added.
    2. Test case: `add pn/John Doe Jun Kai ic/T0123456B p/87654321 p/12345678 a/21 Serangan Road e/john@doe.com u/high d/Dr Tan Ah Beng nk/Mary Doe nkp/87654321 nkr/Mother` (There are 2 phone prefixes, but only one is allowed)
    Expected: No patient record is added to the list. An error message is shown.

6. Adding a duplicated person

    1. Prerequisites: There exist a patient record in the with the same IC number as the person being added.
    2. Test case: `add pn/John Doe Jun Kai ic/T0123456B p/12345678 a/21 Serangan Road e/john@doe.com u/high d/Dr Tan Ah Beng nk/Mary Doe nkp/87654321 nkr/Mother`
    Expected: No patient record is added to the list. An error message is shown.

Deleting patient(s)

1. Deleting a single patient

    1. Prerequisites: List all patients using the `list` command. Ensure there is at least 1 patient in the list for every test case.

    2. Test case: `delete 1`
    Expected: First patient record is deleted from the list. Details of the deleted patient record shown in the success status message.

    3. Test case: `delete`
    Expected: No patient record is deleted. Error details shown indicating empty index field.

    4. Test case: `delete 0`, `delete a`
    Expected: No patient record is deleted. Error details shown indicating invalid command format.

    5. Test case: `delete 100` (assuming index 100 is out of list range)
    Expected: No patient record is deleted. Error details shown indicating invalid patient index.

2. Deleting multiple patients

    1. Prerequisites: List all patients using the `list` command. Ensure there are at least 3 patients in the list for every test case.

    2. Test case: `delete 1,3`
       Expected: First and third patient records are deleted from the list. Details of the deleted patient records shown in the success status message.

    3. Test case: `delete 1-3`
       Expected: First, second and third patient records are deleted from the list. Details of the deleted patient records shown in the success status message.

    4. Test case: `delete 1,1`
       Expected: No patient records are deleted. Error details shown indicating duplicated indices are not allowed.

    5. Test case: `delete 1,100`, `delete 1-100` (assuming index 100 is out of list range)
       Expected: No patient records are deleted. Error details shown indicating invalid patient index.

    6. Test case: `delete 3-1`
       Expected: No patient records are deleted. Error details shown indicating invalid index range.

3. Deleting patients from an empty list

    1. Prerequisites: Delete all patients using the `clear` command. Ensure there are no patients in the list.

    2. Test case: `delete 1`, `delete 1,3`, `delete 1-3`
       Expected: No patient record is deleted. Error details shown indicating that the list is empty.

4. Deleting patient fields

    1. Prerequisites: List all patients using the `list` command. Ensure there is at least 1 patient in the list for every test case.

    2. Test case: `delete 1 s/`
       Expected: The symptoms of the first patient are deleted. Details of the target patient record shown in the success status message.

    3. Test case: `delete 1 s/ n/`
       Expected: The symptoms and notes of the first patient are deleted. Details of the target patient record shown in the success status message.

4. Test case: `delete 1 ic/`
   Expected: No field values are deleted. Error details shown indicating invalid field prefix.

5. Test case: `delete 1 n/notes`
   Expected: No field values are deleted. Error details shown indicating argument for prefix not allowed.

Updating patient(s)

1. Updating a single patient

   1. Prerequisites: List all patients using the `list` command. Ensure there is at least 1 patient in the list.
   2. Test case: `update 1 u/high s/Fever`
      - Expected: The urgency level of the 1st patient is updated to `high`, and their symptoms are replaced with `Fever`. The status message shows a success message confirming the update of the patient.
   3. Test case: `update 100 u/extreme` (assuming index 100 is out of bounds)
      - Expected: Command is rejected. Error details shown indicating an invalid patient index.
   4. Test case: `update 1`
      - Expected: Command is rejected. Error details shown indicating that at least one field to update must be provided.
2. Updating multiple specific patients

   1. Prerequisites: List all patients using the `list` command. Ensure there are at least 3 patients in the list.
   2. Test case: `update 1,3 u/high s/Fever`
      - Expected: The urgency level of the 1st and 3rd patients is updated to `high`, and their symptoms are replaced with `Fever`. The status message shows a success message confirming the update of both patients.
   3. Test case: `update 1,1 u/high`
      - Expected: Command is rejected. Error details shown indicating duplicated indices are not allowed.
   4. Test case: `update 1,100 u/extreme` (assuming index 100 is out of bounds)
      - Expected: Command is rejected. Error details shown indicating an invalid patient index. No patients are updated (verifying the all-or-nothing execution).

Saving data

1. Dealing with corrupted data files

   1. Locate the data file (e.g. `data/addressbook.json`) and open it with a text editor. Corrupt the data by changing the content to an invalid JSON format (e.g. delete a closing bracket).

    2. Re-launch the application.
       Expected: The app should start with an empty address book.

2. Dealing with missing data files

    1. Delete the data file (e.g. `data/addressbook.json`).

    2. Re-launch the application.
       Expected: The app should start with an empty address book.

Appendix: Planned Enhancements

Team size: 6

1. **Make `Command Box` to support multi-line input**: The current implementation of the command box only supports single-line input, which can be limiting for commands that require multiple fields or long inputs. Implementing multi-line input support would allow users to enter more complex commands more easily. We plan to enhance the command box to support multi-line input, allowing users to press `Shift + Enter` to create a new line within the command box and allowing for text wrapping for long commands. A possible implementation approach is to replace the current `TextField` with a `TextArea` that supports multi-line input and text wrapping, while still allowing for efficient command entry. This enhancement would improve the user experience by making it easier to enter and read complex commands without needing to scroll horizontally or use external text editors.

2. **Undo more than 1 step before**: Currently, the undo functionality only allows users to undo the most recent command. This can be limiting if a user wants to undo multiple recent actions. We plan to enhance the undo functionality to allow users to undo multiple steps back in their command history. This could be implemented by maintaining a stack of previous states of the address book, allowing users to navigate back through their command history and restore previous states as needed. This enhancement would provide users with greater flexibility and control over their data, allowing them to easily correct mistakes or revert changes without needing to manually re-enter commands.

3. **Undo support for bulk updates (e.g., `update 1,2 ...`)**: Currently, the undo functionality does not support undoing bulk updates that affect multiple patient records at once. We plan to enhance the undo functionality to allow users to undo bulk updates, restoring all affected patient records to their previous states. This could be implemented by storing a snapshot of the address book before executing a bulk update command, allowing users to revert all changes made by that command with a single undo action. This enhancement would improve the robustness of the application by ensuring that users can easily recover from mistakes made during bulk updates, which can be more complex and error-prone than single-record updates.

4. **Make delete and update command more robust against large index inputs**: Currently, if a user enters a delete or update command with an index that is very large, specifically larger than `Integer.MAX_VALUE`, the application may throw the wrong error message indicating an

invalid command format instead of an invalid patient index. We plan to enhance the input validation for delete and update commands to correctly handle large index inputs by checking if the index exceeds the maximum allowed value and providing an appropriate error message indicating that the patient index is out of bounds. This enhancement would improve the user experience by providing clearer feedback on invalid inputs and preventing confusion caused by misleading error messages. The error message would be: "The person index provided is invalid: Please use an index within the range of `[start_index]` and `[end_index]`."

5. **Make the app more robust against corrupted save files**: Currently, if the save file is corrupted (e.g. contains invalid JSON), the application will load into an empty address book without any warning in the result display,only logging is done: `WARNING: Data file at data\clinicconnect.json could not be loaded. Will be starting with an empty AddressBook.`. This can lead to silent data loss if the user continues to use the application unknowingly and saves the address book. We plan to enhance the application's robustness against corrupted save files by implementing error handling during the loading process. If a corrupted save file is detected, the application will load into an empty address book but will also display a warning message to the user indicating that the save file is corrupted and that a new empty address book has been loaded. A backup of the corrupted save file will also be created with a timestamp in the filename for potential recovery. This enhancement would improve the application's resilience to data corruption and provide users with better awareness of issues with their save files, allowing them to take appropriate action to recover their data if needed. The warning message would be: "Warning: The save file is corrupted and could not be loaded. A new empty address book has been created. A backup of the corrupted save file has been saved as 'address-book-backup_.json'."

6. **Make update index out of bounds error message more informative**: Currently, if a user enters an update command with an index that is out of bounds, the application will display a generic error message indicating an invalid patient index: `The person index provided is invalid`. We plan to enhance the error message for out-of-bounds indices in update commands to provide more specific information about the valid index range. The enhanced error message would indicate the range of valid indices , providing users with clearer guidance on what constitutes a valid index. This enhancement would improve the user experience by providing clearer guidance on valid input values and helping users understand how to correct their command inputs. The enhanced error message would be: "The person index provided is invalid: Please use an index within the range of `[start_index]` and `[end_index]`."